

BỘ CÔNG THƯƠNG
TRƯỜNG CAO ĐẲNG KỸ THUẬT CAO THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
MÔN: KIỂM THỬ PHẦN MỀM

ĐỀ TÀI C
NGHIÊN CỨU VÀ TRIỂN KHAI TỰ ĐỘNG HÓA KIỂM THỬ
VỚI PLAYWRIGHT

Giảng viên hướng dẫn: ThS. Nguyễn Hoàng Việt

Nhóm thực hiện: Nhóm 67

Danh sách thành viên nhóm:

STT	Họ và Tên Sinh Viên	MSSV
1	Trần Văn Ngọc	0306241131
2	Ngô Gia Bảo	0306241090
3	Nguyễn Quốc Đường	0306241102
4	Đặng Duy Lam	0306241125
5	Nguyễn Hoài Linh	0306241126
6	Lê Minh Quân	0306241143
7	Lê Minh Tài	0306241145

TP. HỒ CHÍ MINH, NĂM 2026

LỜI MỞ ĐẦU & TÓM TẮT ĐỀ TÀI

Trong bối cảnh phát triển phần mềm hiện đại với các chu kỳ phát hành liên tục (CI/CD), kiểm thử tự động (Automation Testing) đóng vai trò sống còn trong việc đảm bảo chất lượng, rút ngắn thời gian phản hồi lỗi và giảm thiểu chi phí kiểm thử hồi quy. Đề án này tập trung nghiên cứu chuyên sâu về **Playwright** — nền tảng kiểm thử tự động thế hệ mới được phát triển bởi Microsoft, đồng thời tiến hành đối sánh toàn diện với công cụ thương mại **TestComplete** (SmartBear) cùng các giải pháp kiểm thử phổ biến khác như Selenium WebDriver và Cypress.

Báo cáo được cấu trúc thành 4 chương chính:

1. **Chương 1 — Tổng Quan Lý Thuyết & Cơ Sở Kiến Trúc:** Trình bày lịch sử, kiến trúc đa tiến trình, cơ chế kết nối trực tiếp qua Chrome DevTools Protocol (CDP), cơ chế Auto-waiting và Browser Context Isolation.
2. **Chương 2 — Hướng Dẫn Cài Đặt, Cấu Hình & Bộ Công Cụ SDET:** Hướng dẫn chi tiết quy trình cài đặt môi trường, phân tích cấu hình `playwright.config.ts` và làm chủ bộ tứ công cụ (CLI, Codegen, UI Mode, Trace Viewer).
3. **Chương 3 — Thiết Kế & Thực Nghiệm Kịch Bản Kiểm Thử (Demo):** Hiện thực bộ kịch bản kiểm thử hai tầng (Dual-Engine) gồm 4 ca kiểm thử API trên backend ‘ticket-booking’ (chống đặt trùng vé với Redis Redlock, Idempotency, RFC 9457) và 4 ca kiểm thử Web UI trên hệ thống SauceDemo Swag Labs theo mô hình Page Object Model (POM), kèm cấu hình CI/CD Pipeline.
4. **Chương 4 — Đánh Giá Đối Sánh Công Cụ & Kết Luận:** Xây dựng bảng ma trận so sánh 7 tiêu chí kỹ thuật, phân tích các ranh giới kỹ thuật cứng (Non-goals), đúc kết bài học phòng chống Anti-patterns và đề xuất hướng phát triển.

MỤC LỤC

Lời Mở Đầu & Tóm Tắt Đề Tài	i
1 Tổng Quan Lý Thuyết & Cơ Sở Kiến Trúc	1
1.1 Giới Thiệu Tổng Quan Về Playwright & TestComplete	1
1.1.1 Lịch Sử Ra Đời & Bối Cảnh Công Nghệ	1
1.1.2 Triết Lý Thiết Kế (Code-First vs GUI/Record-Replay)	1
1.2 Cơ Chế Giao Tiếp Qua Chrome DevTools Protocol (CDP)	1
1.2.1 Mô Hình Kết Nối WebSocket Hai Chiều Trực Tiếp	1
1.2.2 So Sánh Cơ Chế Push Event vs Pull/Polling	2
1.3 Cơ Chế Auto-Waiting Thông Minh	2
1.4 Cơ Chế Cô Lập Môi Trường (Browser Context Isolation)	2
2 Hướng Dẫn Cài Đặt, Cấu Hình & Bộ Công Cụ SDET	4
2.1 Quy Trình Cài Đặt Môi Trường	4
2.2 Phân Tích Tập Cấu Hình Trung Tâm ('playwright.config.ts')	4
2.3 Hướng Dẫn Sử Dụng Bộ 4 Công Cụ Hỗ Trợ SDET	4
3 Thiết Kế & Thực Nghiệm Kịch Bản Kiểm Thử	6
3.1 Thiết Kế Kiến Trúc Kiểm Thử Hai Tầng (Dual-Engine Architecture)	6
3.2 Thực Nghiệm Kiểm Thử Tầng API (Target: Backend 'ticket-booking')	6
3.2.1 Ca 1: Vòng Đời Xác Thực & Token Rotation ('AuthService')	6
3.2.2 Ca 2: Tranh Chấp Đặt Vé Đồng Thời & Khóa Phân Tán Redis Redlock	6
3.2.3 Ca 3: Xác Nhận Đơn Vé & Tính Lũy Động (Idempotency Key)	7
3.2.4 Ca 4: Chuẩn Hóa Mã Lỗi RFC 9457 & Ngưỡng Chặn Rate Limiting	7
3.3 Thực Nghiệm Kiểm Thử Tầng Web UI (Target: SauceDemo Swag Labs)	8
3.3.1 Ca 1: Luồng Đặt Hàng Hoàn Chỉnh (POM & COM)	8
3.3.2 Ca 2: Can Thiệp Mạng <code>page.route()</code> & Giả Lập Lỗi	8
3.3.3 Ca 3 & 4: Gỡ Lỗi Bằng Trace Viewer & Kiểm Thử Hồi Quy Trực Quan	8

3.4	Tự Động Hóa Tích Hợp Liên Tục (CI/CD Pipeline)	8
4	Đánh Giá Đối Sánh Công Cụ & Kết Luận	9
4.1	Bảng Ma Trận So Sánh 7 Tiêu Chí Kỹ Thuật	9
4.2	Phân Tích Ưu Điểm, Nhược Điểm & Ranh Giới Kỹ Thuật (Hard Boundaries)	9
4.3	Đúc Kết Bài Học & Phòng Chống Anti-Patterns Trong SDET	10
4.4	Kết Luận & Hướng Phát Triển	10
	Tài Liệu Tham Khảo	11

DANH SÁCH BẢNG

4.1	Bảng ma trận đối sánh kỹ thuật giữa Playwright và các công cụ tự động hóa. . .	9
-----	--	---

CHƯƠNG 1

TỔNG QUAN LÝ THUYẾT & CƠ SỞ KIẾN TRÚC

1.1. Giới Thiệu Tổng Quan Về Playwright & TestComplete

1.1.1. Lịch Sử Ra Đời & Bối Cảnh Công Nghệ

Playwright được Microsoft ra mắt vào năm 2020, được dẫn dắt bởi đội ngũ kỹ sư từng phát triển Puppeteer tại Google. Playwright ra đời nhằm giải quyết triệt để các hạn chế cố hữu của Selenium WebDriver (vốn phụ thuộc vào giao thức HTTP trung gian có độ trễ cao) và Cypress (vốn bị giới hạn bởi môi trường In-browser Iframe). Trong khi đó, TestComplete là phần mềm kiểm thử tự động thương mại được SmartBear phát triển từ năm 1999, hướng đến các doanh nghiệp truyền thống với nhu cầu kiểm thử cả ứng dụng Desktop lẫn Web.

1.1.2. Triết Lý Thiết Kế (Code-First vs GUI/Record-Replay)

Playwright theo đuổi triết lý **Code-First**, hướng tới các kỹ sư SDET (Software Development Engineer in Test) và lập trình viên. Toàn bộ kịch bản, bộ định vị (Locators) và cấu hình đều được viết bằng mã nguồn TypeScript/JavaScript rõ ràng, dễ dàng quản lý phiên bản qua Git. Ngược lại, TestComplete tập trung vào mô hình **Low-Code / Keyword-Driven**, hỗ trợ tính năng Record & Replay mạnh mẽ thông qua giao diện đồ họa GUI nhưng tạo ra các file ảnh xạ đối tượng công kênh khó giải quyết xung đột khi làm việc nhóm.

1.2. Cơ Chế Giao Tiếp Qua Chrome DevTools Protocol (CDP)

1.2.1. Mô Hình Kết Nối WebSocket Hai Chiều Trực Tiếp

Khác với Selenium phải khởi tạo HTTP Request/Response riêng biệt cho từng hành động click hay gõ phím, Playwright thiết lập một kết nối WebSocket nhị phân duy nhất trực tiếp vào Browser Engine qua Chrome DevTools Protocol (CDP) và giao thức mở BiDi (WebDriver BiDirectional).

MÔ HÌNH SO SÁNH GIAO THỨC TRUYỀN THÔNG

1. Mô hình Selenium (HTTP REST Polling):

Test Code → (HTTP POST) → [chromedriver.exe] → (IPC) → [Browser]

2. Mô hình Playwright (Direct WebSocket CDP Push):

Test Code <===== [WebSocket 2 chiều]
===== > [Browser]

1.2.2. So Sánh Cơ Chế Push Event vs Pull/Polling

Nhờ kết nối WebSocket hai chiều, Playwright hoạt động theo mô hình ****Event-Driven (Đẩy sự kiện)****: Trình duyệt sẽ chủ động bắn tín hiệu về Test Runner ngay khi DOM thay đổi hoặc Network Request hoàn tất, triệt tiêu hoàn toàn thời gian chờ thăm dò (Polling) lãng phí của các framework cũ.

1.3. Cơ Chế Auto-Waiting Thông Minh

Hiện tượng kiểm thử không ổn định (Flaky Tests) phần lớn xuất phát từ việc kịch bản kiểm thử cố gắng tương tác khi phần tử chưa sẵn sàng. Playwright tích hợp cơ chế Auto-waiting tự động thực thi 5 bước kiểm tra khả năng hành động (Actionability Checks) trước mọi thao tác:

1. **Attached:** Kiểm tra phần tử đã được đính kèm vào cây DOM hay chưa.
2. **Visible:** Kiểm tra phần tử có hiển thị thực tế trên màn hình (kích thước khác 0, không bị che bởi CSS 'display: none' hoặc 'visibility: hidden').
3. **Stable:** Kiểm tra phần tử đã dừng các hiệu ứng chuyển động (Animation/Transition).
4. **Enabled:** Kiểm tra phần tử không ở trạng thái vô hiệu hóa ('disabled').
5. **Editable:** Kiểm tra phần tử sẵn sàng nhận dữ liệu nhập bàn phím.

1.4. Cơ Chế Cô Lập Môi Trường (Browser Context Isolation)

Playwright xây dựng mô hình phân cấp tiến trình 3 tầng: Browser → BrowserContext → Page. Mỗi BrowserContext là một phiên ẩn danh (Incognito Session) cô lập hoàn toàn

về Cookies, LocalStorage, SessionStorage và Cache. Việc khởi tạo một `BrowserContext` mới chỉ mất vài mili-giây và tiêu tốn rất ít tài nguyên RAM, cho phép thực thi hàng trăm kịch bản kiểm thử song song đa luồng mà không xảy ra hiện tượng rò rỉ trạng thái (State Leakage).

CHƯƠNG 2

HƯỚNG DẪN CÀI ĐẶT, CẤU HÌNH & BỘ CÔNG CỤ SDET

2.1. Quy Trình Cài Đặt Môi Trường

Dự án được khởi tạo trên nền tảng Node.js / Bun và TypeScript. Quy trình cài đặt được thực hiện theo các bước:

```
1 # 1. Khởi tạo dự án Playwright với TypeScript
2 npm init playwright@latest --yes
3
4 # 2. Cài đặt các trình duyệt độc lập (Chromium, Firefox, WebKit)
5 npx playwright install --with-deps chromium firefox webkit
```

Listing 2.1: Lệnh khởi tạo và cài đặt Playwright Test.

2.2. Phân Tích Tập Cấu Hình Trung Tâm ('playwright.config.ts')

Tập 'playwright.config.ts' đóng vai trò là trung tâm điều phối toàn bộ hành vi kiểm thử:

- **Phân tách Multi-Project:** Cấu hình riêng biệt dự án 'api-tests' (chạy không cần browser) và 'ui-tests' (chạy trên Chromium/Firefox).
- **Điều khiển luồng thực thi:** Thiết lập 'fullyParallel: true' chạy song song tối đa CPU, 'retries: 2' trên môi trường CI và 'workers: 4' trên môi trường nội bộ.
- **Thu thập bằng chứng kiểm toán:** Cấu hình 'trace: 'on-first-retry'' và 'screenshot: 'only-on-failure''.

2.3. Hướng Dẫn Sử Dụng Bộ 4 Công Cụ Hỗ Trợ SDET

1. **Playwright CLI:** Thực thi dòng lệnh với các tham số mạnh mẽ như '--grep @smoke' (lọc test case theo tag), '--headed' (chạy hiển thị giao diện) và '--project=api-tests'.
2. **Playwright Codegen (npx playwright codegen <url>):** Tự động ghi lại thao tác của người dùng trên trình duyệt và sinh mã nguồn TypeScript chuẩn mực theo bộ định vị Role-based Locators ('getByRole').

3. **Playwright UI Mode (`npx playwright test -ui`):** Môi trường gỡ lỗi tương tác trực quan thời gian thực, tích hợp Watch Mode (tự động chạy lại khi sửa code) và dòng thời gian trực quan.
4. **Playwright Trace Viewer (`npx playwright show-trace <trace.zip>`):** Công cụ điều tra lỗi hậu kỳ chuyên sâu, hiển thị đồng bộ 4 luồng dữ liệu: DOM Snapshots trước/sau thao tác, Network Waterfall HAR, Console Logs và Screencast video.

CHƯƠNG 3

THIẾT KẾ & THỰC NGHIỆM KỊCH BẢN KIỂM THỬ

3.1. Thiết Kế Kiến Trúc Kiểm Thử Hai Tầng (Dual-Engine Architecture)

Hệ thống áp dụng kiến trúc kiểm thử hai tầng nhằm tối ưu hóa độ bao phủ và tốc độ:

- **Tầng API Automation:** Tương tác trực tiếp với Backend ‘ticket-booking’ qua `APIRequestContext` (Node.js runtime network stack).
- **Tầng Web UI Automation:** Điều khiển giao diện người dùng trên hệ thống thương mại điện tử SauceDemo Swag Labs theo mô hình Page Object Model (POM).

3.2. Thực Nghiệm Kiểm Thử Tầng API (Target: Backend ‘ticket-booking’)

Tầng API được thiết kế với 4 ca kiểm thử trọng điểm nhằm bề gãy và xác thực các cơ chế backend phức tạp:

3.2.1. Ca 1: Vòng Đời Xác Thực & Token Rotation (‘AuthService’)

Kiểm tra luồng đăng ký tài khoản mới → Đăng nhập trích xuất Access Token / Refresh Token → Thực hiện Refresh Token và kiểm định cơ chế **Single-use Token Rotation** (thu hồi Refresh Token cũ ngay sau khi cấp mới) → Thử phát lại (Replay attack) token cũ và xác thực phản hồi HTTP 401 Unauthorized.

3.2.2. Ca 2: Tranh Chấp Đặt Vé Đồng Thời & Khóa Phân Tán Redis Redlock

Mô phỏng tình huống 10 người dùng đồng thời bấm nút đặt cùng một chiếc vé duy nhất tại thời điểm t_0 bằng kỹ thuật bắn bão `Promise.all()`:

```
1 // tests/api/concurrency-race-condition.spec.ts
2 import { test, expect } from '@playwright/test';
3
4 test.describe('API Concurrency - Redis Redlock Race Condition', () => {
5   test('Should allow only 1 booking when 10 requests burst
6     simultaneously', async ({ request }) => {
7     const seatId = 'seat_vip_A12';
```

```
7     const totalRequests = 10;
8
9     // 1. Ban dong thoi 10 requests qua Promise.all()
10    const requests = Array.from({ length: totalRequests }).map((_, idx)
11        =>
12        request.post('/api/v1/bookings/reserve', {
13            data: { seatId, userId: `user_${idx + 1}` },
14        })
15    );
16
17    // 2. Cho toan bo phan hoi
18    const responses = await Promise.all(requests);
19    const statusCodes = responses.map((res) => res.status());
20
21    // 3. Kiem dinh: Dung 1 ma 201 Created va 9 ma 409 Conflict
22    const successCount = statusCodes.filter((c) => c === 201).length;
23    const conflictCount = statusCodes.filter((c) => c === 409).length;
24
25    expect(successCount).toBe(1);
26    expect(conflictCount).toBe(totalRequests - 1);
27    });
```

Listing 3.1: Kịch bản kiểm thử API Concurrency bằng Promise.all().

3.2.3. Ca 3: Xác Nhận Đơn Vé & Tính Lũy Động (Idempotency Key)

Gửi yêu cầu xác nhận đơn vé kèm tiêu đề `Idempotency-Key`. Gửi lại chính xác yêu cầu đó lần thứ hai và xác thực hệ thống trả về đơn vé đã xác nhận (HTTP 200 OK) mà không tạo thêm giao dịch thanh toán hay trừ tiền lần hai.

3.2.4. Ca 4: Chuẩn Hóa Mã Lỗi RFC 9457 & Ngưỡng Chặn Rate Limiting

Gửi payload sai định dạng để kiểm định cấu trúc lỗi theo chuẩn RFC 9457 Problem Details (application/problem+json) bằng Zod Schema. Đồng thời bắn liên tiếp 15 requests vào API đăng nhập để kích hoạt Throttler Guard và xác thực mã lỗi HTTP 429 Too Many Requests kèm header `Retry-After`.

3.3. Thực Nghiệm Kiểm Thử Tầng Web UI (Target: SauceDemo Swag Labs)

3.3.1. Ca 1: Luồng Đặt Hàng Hoàn Chính (POM & COM)

Xây dựng các lớp đối tượng trang (`LoginPage`, `InventoryPage`, `CartPage`, `CheckoutPage`) và thành phần tái sử dụng (`NavbarComponent`). Thực thi trọn vẹn luồng từ đăng nhập → chọn sản phẩm → nhập thông tin → xác nhận đơn hàng thành công.

3.3.2. Ca 2: Can Thiệp Mạng `page.route()` & Giả Lập Lỗi

Sử dụng tính năng Network Interception `page.route()` để chặn bắt request tải ảnh sản phẩm và phản hồi mã lỗi HTTP 500 `Internal Server Error`. Kiểm tra giao diện ứng dụng hiển thị ảnh fallback hợp lệ mà không bị sập trang. Đồng thời kiểm thử trường hợp đăng nhập bằng tài khoản bị khóa (`locked_out_user`).

3.3.3. Ca 3 & 4: Gỡ Lỗi Bằng Trace Viewer & Kiểm Thử Hồi Quy Trực Quan

Kiểm thử tài khoản có độ trễ mạng (`performance_glitch_user`), kích hoạt Trace Viewer để phân tích biểu đồ thác nước mạng (HAR Waterfall). Áp dụng tính năng `toHaveScreenshot()` với kỹ thuật che vùng động (Masking) để kiểm thử sai lệch điểm ảnh giao diện (Visual Regression).

3.4. Tự Động Hóa Tích Hợp Liên Tục (CI/CD Pipeline)

Thiết lập tệp cấu hình `.github/workflows/playwright.yml` tự động kích hoạt kiểm thử mỗi khi có Pull Request. Sử dụng Docker Container `mcr.microsoft.com/playwright` và tự động đính kèm HTML Test Report vào GitHub Artifacts khi hoàn tất.

CHƯƠNG 4

ĐÁNH GIÁ ĐỐI SÁNH CÔNG CỤ & KẾT LUẬN

4.1. Bảng Ma Trận So Sánh 7 Tiêu Chí Kỹ Thuật

Nhằm cung cấp góc nhìn toàn diện cho việc lựa chọn công cụ kiểm thử tự động, bảng dưới đây đối sánh chi tiết giữa Playwright và các công cụ tiêu biểu:

Tiêu Chí	Playwright	TestComplete	Selenium 4	Cypress
Kiến trúc điều khiển	WebSocket / CDP trực tiếp	OS Native Hook	HTTP REST Server	In-browser Iframe
Bản quyền & Chi phí	Mã nguồn mở (100% Free)	Thương mại (Bản quyền cao)	Mã nguồn mở (100% Free)	Mã nguồn mở + Thu phí Cloud
Tốc độ thực thi	Siêu nhanh (< 5 ms)	Trung bình	Chậm (HTTP latency)	Nhanh đơn tab
Hỗ trợ Multi-tab	Hỗ trợ Native	Hỗ trợ	Hỗ trợ	Rất hạn chế
Kiểm thử API	Có (APIRequest)	Có module riêng	Không có	Có (cy.request)
Cơ chế Auto-waiting	Tự động 5 bước	Cấu hình tĩnh	Thủ công	Tự động cơ bản
Đối tượng phù hợp	Kỹ sư SDET, Lập trình viên	Doanh nghiệp, Manual QA	Hệ thống lớn kế thừa	Lập trình viên Frontend

Bảng 4.1: Bảng ma trận đối sánh kỹ thuật giữa Playwright và các công cụ tự động hóa.

4.2. Phân Tích Ưu Điểm, Nhược Điểm & Ranh Giới Kỹ Thuật (Hard Boundaries)

- **Ưu điểm vượt trội:** Tốc độ thực thi nhanh, cơ chế Auto-waiting triệt tiêu Flaky test, hỗ trợ kiểm thử API tích hợp sẵn và bộ công cụ Trace Viewer mạnh mẽ.
- **Ranh giới kỹ thuật cứng (Non-goals):** Playwright không hỗ trợ kiểm thử đồ họa Canvas/WebGL nội bộ, không sinh ra để vượt các hệ thống WAF/Bot Detection (như Cloudflare), và không thể thay thế các công cụ tạo tải phân tán (Load Testing như k6/JMeter).

4.3. Đúc Kết Bài Học & Phòng Chống Anti-Patterns Trong SDET

- Tuyệt đối không sử dụng lệnh chờ cưỡng bức `page.waitForTimeout()` gây chậm và giòn kịch bản.
- Tuân thủ quy tắc đóng gói Page Object Model: Không nhúng các câu lệnh kiểm định (`expect`) vào bên trong phương thức của Page Object.
- Tách biệt dữ liệu kiểm thử và áp dụng kỹ thuật Dynamic Test Fixtures để triệt tiêu rác cơ sở dữ liệu.

4.4. Kết Luận & Hướng Phát Triển

Đồ án đã nghiên cứu và hiện thực thành công khung kiểm thử tự động hai tầng với Playwright. Kết quả thực nghiệm chứng minh Playwright đáp ứng hoàn hảo các yêu cầu kiểm định khắt khe của phần mềm hiện đại, mang lại hiệu quả cao cho doanh nghiệp và kỹ sư kiểm thử phần mềm.

TÀI LIỆU THAM KHẢO

1. Microsoft Corporation, *Playwright Documentation & API Reference*, <https://playwright.dev/docs/intro>, 2026.
2. SmartBear Software, *TestComplete Documentation*, <https://support.smartbear.com/testcomplete/docs/>, 2026.
3. W3C, *WebDriver BiDirectional Protocol Specification*, World Wide Web Consortium, 2025.
4. Martin Fowler, *Page Object Model Pattern*, <https://martinfowler.com/bliki/PageObject.html>, 2013.
5. IETF, *RFC 9457: Problem Details for HTTP APIs*, Internet Engineering Task Force, 2023.